

Python for Quants

Expanded Reference Edition

This is the expanded reference edition of the Level 1 topic that opens the curriculum’s Python track: **Python Fundamentals → Scientific Python → Financial Python → Professional Python**. Each stage keeps its original learning-position summary and then goes substantially deeper: concept walkthroughs explaining the intuition, fully worked examples with runnable code, and reference tables of the key tools. The four stages are cumulative — each one is the working assumption behind the next — so they are presented in the order a quant researcher actually acquires them.

Table of Contents

Right-click and choose “Update Field” (or “Update entire table”) after opening in Word to populate page numbers.

1. Python Fundamentals (Level 1)	3
1.1 Learning Position at a Glance	3
1.2 Concept Walkthrough	3
1.3 Worked Example — Mutable Default Argument Bug	4
1.4 Worked Example — A Small Position-Sizing Module with Custom Exceptions	5
1.5 Reference Table — Core Language Building Blocks	5
2. Scientific Python (Level 1)	6
2.1 Learning Position at a Glance	6
2.2 Concept Walkthrough	6
2.3 Worked Example — Vectorising a Rolling Calculation	7
2.4 Worked Example — Fitting a Distribution with SciPy	8
2.5 Reference Table — Core Scientific-Python Tools	8
3. Financial Python (Level 1)	10
3.1 Learning Position at a Glance	10
3.2 Concept Walkthrough	10
3.3 Worked Example — Vectorised OHLCV Resampling and Rolling Volatility	12
3.4 Worked Example — A Look-Ahead Bug and Its Fix	13
3.5 Reference Table — Financial-Python Operations	13
4. Professional Python (Level 1)	14
4.1 Learning Position at a Glance	14
4.2 Concept Walkthrough	14

4.3 Worked Example — Adding Types, Logging and a Test to the Position-Sizer	15
4.4 Worked Example — Concurrent API Calls with asyncio	16
4.5 Reference Table — Professional-Python Practices	16

1. Python Fundamentals (Level 1)

Covers: Variables · data types · lists · dicts · tuples · sets · functions · classes · exceptions · modules

► INTUITION FIRST

A variable is just a name pointing at a value — nothing is copied until you ask for it. Every “why did my other list change too” bug traces back to that one fact: lists, dicts and sets are mutable and shared by reference, while numbers, strings and tuples are not. Learn that distinction now and half of the confusing bugs you’ll hit later never happen.

1.1 Learning Position at a Glance

- **What it is:** The core language: the built-in data types, control flow, functions, classes and error handling that every later library sits on top of.
- **Why it matters:** NumPy, pandas and every trading system are ordinary Python underneath. Shaky fundamentals resurface as mysterious bugs three layers up — a mutable default argument or a shared-reference list corrupts a backtest in a way that looks like a data problem.
- **Prerequisites:** None. This is the true starting point of the Python track.
- **Connects to Research:** Clean functions and classes are what let a notebook experiment turn into a reusable research module without a rewrite.
- **Connects to Development:** Exception handling and module structure are the first two habits professional codebases are graded on.
- **Connects to Trading:** A live trading system is fundamentally functions and classes calling each other under time pressure; there is no separate “trading Python” to learn later.
- **Practical exercise:** Write a small position-sizing module: a function that takes account equity, a risk percentage and a stop distance and returns a share count, plus a custom exception raised when the stop distance is zero or negative.
- **Common beginner mistakes:**
 - Using a mutable object (a list or dict) as a default function argument, which is created once and silently shared across every call.
 - Catching every exception with a bare `except:`, which hides real bugs (typos, wrong types) alongside the error you meant to catch.
- **Advanced extensions:** Decorators, context managers (with) and generators are Level 1 vocabulary that pay off heavily once async and performance work begins (Section 4).
- **Professional applications:** Config objects, custom exception hierarchies and small utility modules are the shared vocabulary of every production codebase.

1.2 Concept Walkthrough

Python's built-in containers split cleanly along one axis: mutability. `list`, `dict` and `set` can be changed in place after creation; `tuple`, `str` and numeric types cannot. This single property explains a whole family of beginner bugs. When you write `b = a` for a list `a`, both names now point at the same underlying object —

mutating `b` mutates `a` too, because there was never a second list, only a second name. The fix is an explicit copy (`a.copy()` or `list(a)`) whenever independence is required.

The same reference-sharing trap hides inside default arguments. Python evaluates a function's default argument values exactly once, when the `def` statement runs — not on every call. A default of `[]` is therefore one specific list object, shared and silently accumulated across every call that doesn't supply its own argument. The idiomatic fix is a default of `None`, with the real empty container created fresh inside the function body.

Exceptions exist so that a failure can be handled at the level that actually knows what to do about it, rather than checked after every single line. A well-designed function raises a specific, meaningfully named exception (a custom subclass of `Exception`) rather than returning `None` or a sentinel value on failure — a `None` that silently flows downstream into a calculation is exactly how a bad price or a missing fill turns into a wrong number three steps later with no error message anywhere near the real cause.

Modules and packages are the mechanism for turning a folder of scripts into something importable and testable. A file becomes a module the moment another file imports it; a folder becomes a package the moment it contains an `__init__.py` (or, in modern Python, simply by being importable as a namespace package). Getting this right early avoids the classic notebook failure mode: months of research logic that only exists as copy-pasted cells and can never be reused, tested, or deployed.

1.3 Worked Example — Mutable Default Argument Bug

- A naive order-log function: `def log_fill(fill, book=[]): book.append(fill); return book`
- Called three times with different fills, the same list keeps growing across calls, because `[]` was created once when the function was defined — all three “independent” order books are actually the same object.
- The fix: `def log_fill(fill, book=None): book = book if book is not None else [];`
`book.append(fill); return book`
- Takeaway: any time a default argument is a list, dict or set, ask whether it should instead be `None` with the container created inside the function body.

```
# Buggy version – book accumulates across unrelated calls
def log_fill(fill, book=[]):
    book.append(fill)
    return book

log_fill("BUY 100 AAPL")    # ['BUY 100 AAPL']
log_fill("SELL 50 MSFT")   # ['BUY 100 AAPL', 'SELL 50 MSFT']  <- unexpected

# Fixed version – a fresh list every call
def log_fill(fill, book=None):
    book = book if book is not None else []
    book.append(fill)
    return book
```

1.4 Worked Example — A Small Position-Sizing Module with Custom Exceptions

```
class RiskError(Exception):
    """Raised when a risk input is invalid."""

def position_size(equity, risk_pct, entry, stop):
    stop_distance = abs(entry - stop)
    if stop_distance <= 0:
        raise RiskError("stop distance must be positive")
    risk_amount = equity * risk_pct
    return int(risk_amount / stop_distance)

shares = position_size(equity=100_000, risk_pct=0.01, entry=50.20, stop=49.70)
# shares == 2000
```

This shape — a custom exception plus a function that raises early on bad input — is the pattern that recurs everywhere in Sections 3 and 4: validate the input, fail loudly and specifically, and never let a silent None or a wrong-shaped default reach a P&L calculation.

1.5 Reference Table — Core Language Building Blocks

Concept	Example	Notes
Mutable vs. immutable	list, dict, set vs. tuple, str, int	Mutable containers are shared by reference; copy explicitly when independence is needed.
Function	def f(x, y=1): return x + y	Default arguments are evaluated once, at definition time — never default to a mutable object.
Class	class Position: def __init__(self, qty): self.qty = qty	Bundles data (attributes) and behaviour (methods) into one object.
Exception handling	try / except SpecificError / finally	Catch the narrowest exception that applies; avoid bare except:.
Custom exception	class RiskError(Exception): pass	Gives calling code something specific to catch, rather than a generic ValueError.
Module / package	import mymodule / from mypkg import tools	A file is a module; a folder of modules is a package.
Comprehension	[x**2 for x in values if x > 0]	Idiomatic, faster than an equivalent explicit loop with .append().

2. Scientific Python (Level 1)

Covers: NumPy · pandas · SciPy · Matplotlib · Plotly

► INTUITION FIRST

NumPy and pandas exist because a Python for-loop pays a fixed tax on every single element — type-checking, function-call overhead — while a vectorised operation pays that tax once for the whole array. “Vectorising” is not a style preference; it is the difference between an operation that runs once in compiled C and one that re-enters the Python interpreter a million times.

2.1 Learning Position at a Glance

- **What it is:** The array- and table-based computing stack: NumPy arrays, pandas Series/DataFrames, SciPy's statistical and optimisation routines, and Matplotlib/Plotly for visualisation.
- **Why it matters:** This is the daily working tool for signal construction, backtesting and research — fluency here directly determines research iteration speed.
- **Prerequisites:** Python Fundamentals (Section 1).
- **Connects to Research:** Every research notebook is, mechanically, a sequence of vectorised NumPy/pandas transformations applied to a DataFrame.
- **Connects to Development:** SciPy-backed and NumPy-backed libraries (scikit-learn, statsmodels) all assume fluent array manipulation as a prerequisite.
- **Connects to Trading:** Real-time risk and signal calculations reuse the exact same vectorised operations as research, just on a rolling live feed instead of a static history.
- **Practical exercise:** Given a DataFrame of daily prices for 20 tickers, compute log returns, a 20-day rolling volatility, and each ticker's rank by that volatility — without writing a single explicit for-loop over rows.
- **Common beginner mistakes:**
 - Iterating over DataFrame rows with `.iterrows()` instead of a vectorised column operation — often 50–100x slower on realistic data sizes.
 - Chained indexing (`df[df.x > 0]['y'] = 1`) that triggers pandas' `SettingWithCopyWarning` and may silently fail to modify the original DataFrame.
- **Advanced extensions:** GPU-accelerated arrays (CuPy) and just-in-time compilation (Numba) belong to Level 4 — Performance Engineering.
- **Professional applications:** Signal libraries, feature pipelines and backtest engines are all built as layered pandas/NumPy transformations.

2.2 Concept Walkthrough

A NumPy array is a fixed-type, contiguous block of memory, which is what makes vectorised operations fast: `a + b` for two arrays runs as a single tight loop in compiled C, with no per-element Python overhead. An explicit Python for loop over the same arrays pays the interpreter's per-iteration cost — type dispatch, function-call overhead, bytecode execution — on every single element. This gap widens, not

narrows, as data size grows, which is why “it's fine on my small test set” is one of the most common false signals in quant code review.

pandas builds a labelled, heterogeneous table on top of NumPy: a DataFrame is essentially a dictionary of NumPy arrays sharing a common row index. That index is doing more work than it looks — it is what lets `df['price'] - df['price'].shift(1)` line up two columns correctly even after filtering, sorting or merging has reordered the rows. Operations that ignore the index (mixing `.values` arrays with differently-ordered DataFrames) are a common source of subtle misalignment bugs.

SciPy sits one layer above NumPy, supplying the numerical routines that would otherwise need to be hand-implemented: optimisation (`scipy.optimize`), statistical distributions and tests (`scipy.stats`), and linear algebra (`scipy.linalg`). In practice, a quant reaches for SciPy the moment a calculation needs more than arithmetic — fitting a distribution to returns, solving for an implied volatility, or minimising a portfolio objective.

Matplotlib and Plotly serve different points in the research workflow rather than competing directly. Matplotlib is the default for static, publication-quality figures embedded in a report or saved to disk. Plotly's payoff is interactivity — zooming into a specific drawdown, hovering to read exact OHLCV values — which matters most while a signal is still being explored, not once its behaviour is settled and being written up.

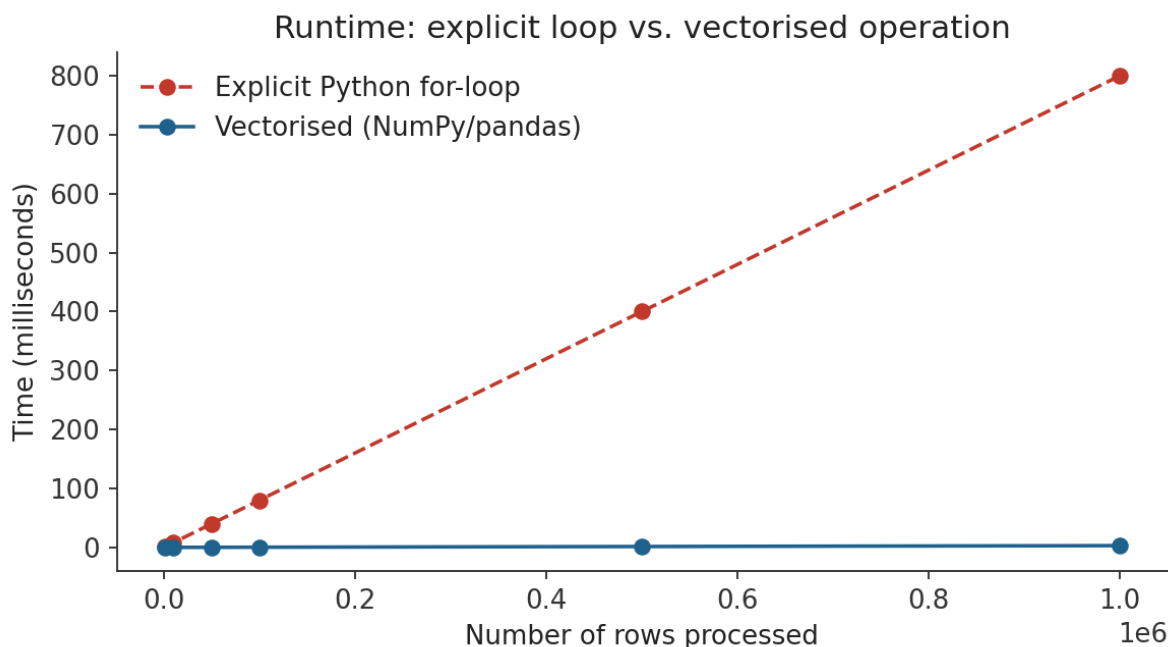


Figure 2.1 — Runtime grows linearly for both approaches, but the vectorised operation's per-element constant is orders of magnitude smaller, because the loop never re-enters the Python interpreter.

2.3 Worked Example — Vectorising a Rolling Calculation

- Given a Series of 500,000 daily-frequency prices, the task is to compute a 20-period rolling mean.
- Loop version: `[sum(prices[i-19:i+1])/20 for i in range(19, len(prices))]` — re-sums the whole window from scratch at every step and pays Python-loop overhead 500,000 times.
- Vectorised version: `prices.rolling(20).mean()` — pandas computes this with an incremental algorithm in compiled code, touching each element a constant number of times.
- Takeaway: the vectorised call is not just shorter — it uses a fundamentally more efficient algorithm under the hood, on top of avoiding interpreter overhead.

```
import pandas as pd, numpy as np

prices = pd.Series(100 + np.cumsum(np.random.normal(0, 0.5, 500_000)))

# Vectorised – idiomatic and fast
rolling_mean = prices.rolling(window=20).mean()

# Avoid: an explicit loop recomputing the sum every step
# rolling_mean_slow = [prices[i-19:i+1].mean() for i in range(19, len(prices))]
```

2.4 Worked Example — Fitting a Distribution with SciPy

```
from scipy import stats
import numpy as np

returns = np.random.normal(0.0004, 0.012, 2000) # daily log returns

# Fit a Student-t distribution (fatter tails than Normal, typical of returns)
df, loc, scale = stats.t.fit(returns)

# 1-day 99% VaR implied by the fitted distribution
var_99 = stats.t.ppf(0.01, df, loc, scale)
```

Fitting a heavier-tailed distribution instead of assuming Normal returns is a one-line change with SciPy, but it directly affects any downstream VaR or option-pricing calculation — exactly the kind of assumption that belongs in Section 3's data-quality discipline.

2.5 Reference Table — Core Scientific-Python Tools

Library	Typical Use	Key Object / Call
NumPy	Fast numerical arrays and vectorised math	<code>np.array</code> , elementwise <code>+</code> , <code>*</code> , <code>np.where</code>
pandas	Labelled tabular data, time series	<code>pd.DataFrame</code> , <code>.rolling()</code> , <code>.groupby()</code>
SciPy	Optimisation, statistics, linear algebra	<code>scipy.optimize.minimize</code> , <code>scipy.stats</code>
Matplotlib	Static, publication-quality charts	<code>plt.plot</code> , <code>fig.savefig</code>
Plotly	Interactive, explorable charts	<code>plotly.express</code> , <code>fig.show()</code>

Library	Typical Use	Key Object / Call
Vectorisation	Replacing an explicit loop with an array op	series.rolling(20).mean() vs. a Python loop

3. Financial Python (Level 1)

Covers: OHLCV handling · returns · resampling · rolling calculations · vectorisation · data cleaning

► INTUITION FIRST

Every quant bug that silently corrupts results, rather than crashing loudly, traces back to time. A rolling window that accidentally includes the current, not-yet-closed bar; a forward-fill that carries yesterday's price into a holiday it shouldn't — both look-ahead bugs are invisible in a backtest's output and only show up as a live strategy that mysteriously underperforms its historical simulation.

3.1 Learning Position at a Glance

- **What it is:** The financial-data layer built on top of Scientific Python: representing OHLCV bars, computing returns correctly, resampling between frequencies, and cleaning real (messy, gappy) market data.
- **Why it matters:** Fast, correct OHLCV and rolling-window handling underlies every live signal and risk calculation; a silent look-ahead bug corrupts a backtest without ever raising an error.
- **Prerequisites:** Python Fundamentals; Scientific Python (Sections 1–2).
- **Connects to Research:** Signal construction and backtesting are, mechanically, a pipeline of resampling, return calculation and rolling-window feature extraction applied to OHLCV data.
- **Connects to Development:** Data-cleaning logic (gap handling, duplicate timestamps, corporate actions) has to be encoded once, correctly, and reused everywhere — not re-derived ad hoc in every notebook.
- **Connects to Trading:** A live signal is the exact same rolling-window calculation as the backtest, evaluated one new bar at a time; any mismatch between the two is a source of live/backtest divergence.
- **Practical exercise:** Given raw tick data, build a vectorised (no explicit Python loops) pipeline that resamples to 1-minute OHLCV bars and computes rolling 20-bar realised volatility.
- **Common beginner mistakes:**
 - Using explicit for-loops over pandas Series/DataFrames instead of vectorised operations.
 - Silent look-ahead bugs from mis-aligned rolling windows or unintentional forward-fill.
- **Advanced extensions:** Async programming and performance optimisation (Numba/Cython) belong to Level 4 — Performance Engineering.
- **Professional applications:** Backtest engines, live signal pipelines and risk dashboards are all built on this exact OHLCV-handling foundation.

3.2 Concept Walkthrough

An OHLCV bar (Open, High, Low, Close, Volume) is a lossy compression of every tick that occurred in a time window. Resampling is the operation that produces it: group raw ticks by time bucket, then take the first price as Open, the max as High, the min as Low, the last as Close, and the sum of traded size as

Volume. In pandas this is `df.resample('1min').agg({'price': 'ohlc', 'size': 'sum'})` — a single vectorised call replacing what would otherwise be a hand-written loop over groups of ticks.

Returns are where sign conventions and off-by-one errors hide. A simple return is $(P_1 - P_0) / P_0$, computed in pandas as `prices.pct_change()`; a log return is $\ln(P_1 / P_0)$, computed as `np.log(prices).diff()`. The critical discipline is alignment: a return at time t must be computed from the price at t and the price strictly before t , never from a price that wasn't yet known at t . `shift(1)` moves data forward in time correctly; getting the sign of the shift backwards is the single most common source of look-ahead bias in a first backtest, because it makes the strategy appear to know the future.

Rolling-window calculations extend this same alignment discipline. `series.rolling(20).std()` at row t uses rows $t-19$ through t inclusive — it does include the current bar. If “current bar” has not actually closed yet in a live system, that rolling calculation is silently using information from the future relative to when a live signal would actually fire. This is why backtests and live systems must share literally the same rolling-window code path, evaluated bar-by-bar in production, rather than two independently written implementations that can quietly drift apart.

Real market data is never clean: duplicate timestamps from multiple exchanges, gaps over weekends and holidays, and stale quotes all appear. `fillna(method='ffill')` is the standard tool for gaps — but applied carelessly across a weekend or halt, it manufactures a flat, zero-volatility period that never happened, understating any volatility measure computed over it. Cleaning decisions (how to handle a halt, a stock split, a missing bar) are modelling assumptions, not neutral defaults, and should be made explicitly rather than inherited silently from whatever a library does by default.

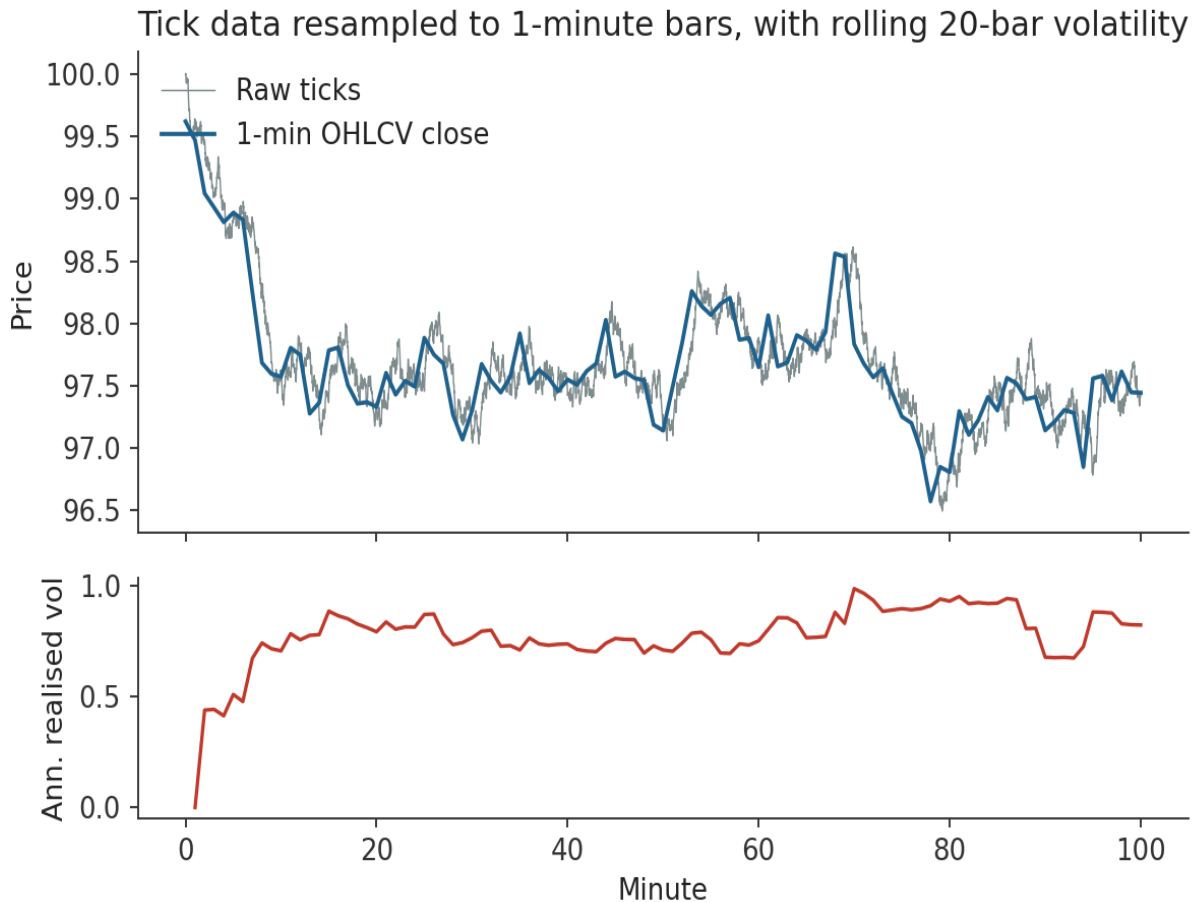


Figure 3.1 — Raw ticks resampled into 1-minute OHLCV bars (top), and the 20-bar annualised realised volatility computed from those bars' log returns (bottom).

3.3 Worked Example — Vectorised OHLCV Resampling and Rolling Volatility

- Raw input: a DataFrame of individual trade ticks with a timestamp index and a price column, arriving at irregular intervals.
- Step 1 — resample to 1-minute bars: `bars = ticks['price'].resample('1min').ohlc()`
- Step 2 — compute log returns of the close: `log_ret = np.log(bars['close']).diff()`
- Step 3 — rolling 20-bar annualised realised volatility: `vol = log_ret.rolling(20).std() * np.sqrt(252 * 390)`
- No explicit Python loop appears anywhere in the pipeline — every step is a single vectorised call across the whole Series.

```
import pandas as pd, numpy as np

# ticks: DataFrame with a DatetimeIndex and a 'price' column
bars = ticks['price'].resample('1min').ohlc()
bars['volume'] = ticks['size'].resample('1min').sum()
```

```
log_ret = np.log(bars['close']).diff()
```

```
ANNUALISATION = np.sqrt(252 * 390) # 390 one-minute bars per trading day  
bars['realised_vol_20'] = log_ret.rolling(window=20).std() * ANNUALISATION
```

3.4 Worked Example — A Look-Ahead Bug and Its Fix

- Buggy signal: `signal = (prices > prices.rolling(20).mean()).astype(int)` used directly as the position for the same bar's return.
- The rolling mean at bar `t` includes the price at bar `t` itself — so the signal is comparing today's close to an average that already contains today's close, and then trading on that same bar's return.
- Fix — shift the signal forward one bar before pairing it with a return: `position = signal.shift(1); strategy_return = position * simple_return`
- Takeaway: a signal computed from bar `t` can only be acted on starting at bar `t+1` — any backtest that pairs a signal with the same bar's return is, structurally, trading on information from the future.

3.5 Reference Table — Financial-Python Operations

Operation	pandas / NumPy Call	Pitfall to Avoid
Resample to OHLCV	<code>prices.resample('1min').ohlc()</code>	Choosing the wrong bar boundary convention (<code>label='left'</code> vs <code>'right'</code>).
Simple return	<code>prices.pct_change()</code>	Adding simple returns across time instead of compounding them.
Log return	<code>np.log(prices).diff()</code>	Mixing log and simple returns in the same calculation.
Rolling statistic	<code>series.rolling(20).std()</code>	Forgetting the window includes the current row — shift before trading on it.
Forward alignment	<code>signal.shift(1)</code>	Omitting the shift, causing a look-ahead bug that is invisible in backtest output.
Gap filling	<code>series.fillna(method='ffill')</code>	Forward-filling across a real gap (holiday, halt) manufactures fake zero-volatility bars.

4. Professional Python (Level 1)

Covers: Virtual environments · package management · type hints · logging · config · testing · docs · Git · APIs · async · performance

► INTUITION FIRST

“It works on my machine” is a dependency-management problem, not a mystery. A virtual environment pins exactly which package versions your code was written against, so that the same code run six months later — or by someone else — behaves identically. Every other practice in this section (types, tests, logging) exists for the same reason: to make behaviour reproducible and failures legible instead of silent.

4.1 Learning Position at a Glance

- **What it is:** The software-engineering practices that turn a working script into something a team can rely on: isolated environments, type hints, logging, configuration, automated tests, documentation, version control, API integration, async I/O and basic performance profiling.
- **Why it matters:** These practices are exactly the Level 4 software-engineering track applied early — research code that will be reused, reviewed or handed off needs them from day one, not retrofitted later.
- **Prerequisites:** Python Fundamentals; Scientific Python; Financial Python (Sections 1–3).
- **Connects to Research:** A tested, typed research module can be trusted and reused by a colleague without them re-reading every line — an untested one cannot.
- **Connects to Development:** This section is the entry point to the full Level 4 software-engineering track (packaging, CI, code review discipline).
- **Connects to Trading:** A live trading system that logs its decisions and fails loudly on bad config is debuggable at 2 a.m.; one that doesn't, isn't.
- **Practical exercise:** Take the position-sizing module from Section 1: add type hints to its signature, a docstring, a logging call on every raised exception, and a pytest test that checks it raises `RiskError` on a zero stop distance.
- **Common beginner mistakes:**
 - Installing packages globally instead of per-project, so that upgrading one project's dependency silently breaks another.
 - Writing code with no automated tests, so that a “small” refactor has no way to confirm it didn't change behaviour.
- **Advanced extensions:** Async programming and performance optimisation (Numba/Cython, profiling, parallelism) belong fully to Level 4 — Performance Engineering; Level 1 only needs to recognise when they'd apply.
- **Professional applications:** Every production quant codebase is built from exactly these pieces: isolated environments, typed and tested modules, structured logging, and version-controlled history.

4.2 Concept Walkthrough

A virtual environment (venv or conda) is a private, isolated copy of the Python interpreter and its installed packages, scoped to one project. Without one, every project on a machine shares a single global set of package versions — upgrading pandas for one project can silently break another that assumed an older version's behaviour. A `requirements.txt` or `pyproject.toml` then records exactly which versions were used, so the environment itself is reproducible by anyone, including a future version of the same person.

Type hints (`def size(equity: float, risk_pct: float) -> int:`) don't change how Python runs — they are not enforced at runtime — but they let a static checker (mypy) and an editor catch a whole class of “passed a string where a number was expected” bugs before the code ever runs, and they double as always-up-to-date documentation of what a function expects and returns.

Logging replaces `print()` as the professional tool for understanding what a running system did, because it carries severity levels (debug, info, warning, error), timestamps, and a destination that can be a file or a monitoring system rather than a terminal that disappears on restart. A live system that logs every trade decision and every caught exception is debuggable after the fact; one that only prints to a console that nobody was watching is not.

Automated tests (pytest) exist to answer one question cheaply and repeatedly: “does this code still do what it's supposed to, after this change?” A test suite is what makes refactoring safe — without one, every change to shared code is a leap of faith that nothing downstream broke. In a quant context, a test that checks a known price series produces a known, hand-verified return or rolling-volatility figure is the difference between “I believe this is right” and “I can prove this is right, and I'll know immediately if it stops being right.”

Git and API integration round out the professional toolkit: version control gives every change an author, a timestamp and a reversible history, which matters enormously when a research result needs to be reproduced exactly as it was six months ago. Working with external APIs (market data vendors, execution venues) means handling authentication, rate limits and network failures as first-class concerns — not edge cases to patch in later — and `async/await` is the tool for doing many such I/O-bound API calls concurrently without spawning a thread per request.

4.3 Worked Example — Adding Types, Logging and a Test to the Position-Sizer

```
import logging

logger = logging.getLogger(__name__)

class RiskError(Exception):
    """Raised when a risk input is invalid."""

def position_size(equity: float, risk_pct: float, entry: float, stop: float) -> int:
```

```

"""Return a share count sized to risk `risk_pct` of `equity` on this trade."""
stop_distance = abs(entry - stop)
if stop_distance <= 0:
    logger.error("invalid stop distance: entry=%s stop=%s", entry, stop)
    raise RiskError("stop distance must be positive")
risk_amount = equity * risk_pct
return int(risk_amount / stop_distance)

```

```

# test_position_size.py
import pytest
from sizing import position_size, RiskError

def test_normal_case():
    assert position_size(100_000, 0.01, 50.20, 49.70) == 2000

def test_zero_stop_distance_raises():
    with pytest.raises(RiskError):
        position_size(100_000, 0.01, 50.20, 50.20)

```

Nothing about the function's logic changed from Section 1 — what changed is that its behaviour is now documented (the docstring and type hints), observable in production (the log line), and provably correct against known cases (the two tests). This is the entire content of “professional Python”: the same code, made trustworthy.

4.4 Worked Example — Concurrent API Calls with asyncio

```

import asyncio, aiohttp

async def fetch_quote(session, symbol: str) -> dict:
    async with session.get(f"https://api.example.com/quote/{symbol}") as resp:
        return await resp.json()

async def fetch_all(symbols: list[str]) -> list[dict]:
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_quote(session, s) for s in symbols]
        return await asyncio.gather(*tasks)

quotes = asyncio.run(fetch_all(["AAPL", "MSFT", "GOOG", "AMZN"]))

```

Fetching four quotes sequentially waits on four separate network round-trips; `asyncio.gather` issues all four requests concurrently and waits only as long as the slowest one — the same pattern scales to hundreds of symbols with no per-request thread overhead.

4.5 Reference Table — Professional-Python Practices

Practice	Tool / Syntax	What It Buys You
Isolated environment	<code>python -m venv .venv; pip install -r requirements.txt</code>	Reproducible, project-scoped dependency versions.

Practice	Tool / Syntax	What It Buys You
Type hints	<code>def f(x: float) -> int:</code>	Editor and static-checker (mypy) catch type bugs before runtime.
Logging	<code>logging.getLogger(__name__).info(...)</code>	Timestamped, severity-tagged, persistent record of what ran.
Config management	<code>.env</code> / <code>config.yaml</code> + a settings loader	Environment-specific values kept out of source code.
Testing	<code>pytest</code> , <code>assert</code> , <code>pytest.raises(...)</code>	Cheap, repeatable proof that a change didn't break behaviour.
Documentation	Docstrings + Sphinx / makedocs	Usage information that lives next to the code and stays current.
Version control	<code>git commit</code> , <code>git branch</code> , pull requests	Full, reversible, attributed history of every change.
Async I/O	<code>async def</code> / <code>await</code> / <code>asyncio.gather</code>	Many concurrent network calls without one thread per call.